

网络基础知识和8266实战

本讲义由[Lucas-Song_\(Lucas Song\)](#)在2025/6/27在 [Obsidian](#)中完成。结合Table of Contents、Better ExportPDF插件和Advanced Tables插件。

Copyright © 2025 [Lucas-Song-zero_\(Lucas Song\)](#).

This article is licensed under the MIT License.

本讲义中使用的所有代码均在本人的ESP8266开发板上通过编译上传和功能测试。

目录

[0. 开始之前](#)

[1. 网络基本知识](#)

- [1.1 以太网\(链路层\)](#)
- [1.2 IP\(网络层\)](#)
 - [1.2.1 公网地址和私网地址](#)
 - [1.2.2 端口\(Port\)](#)
- [1.3 TCP和UDP\(传输层\)](#)
 - [1.3.1 UDP简介](#)
 - [1.3.2 TCP简介](#)
- [1.4 HTTP\(S\)\(应用层\)](#)
 - [1.4.1 HTTP\(超文本传输协议\)](#)
 - [1.4.2 当我们使用域名的时候，背后发生了什么？](#)
 - [1.4.2.1 如何在《地图》上找到一个域名](#)
 - [1.4.2.2 How about the Machines?](#)
 - [1.4.3 HTTP协议如何实现通信的？](#)
 - [1.4.3.1 HTTP请求格式](#)
 - [1.4.3.2 HTTP响应格式](#)
 - [1.4.3.3 8266实例](#)
 - [1.4.4 这个S到底意味着什么？带来了哪些变化？](#)

[2. HTML & CSS & JavaScript](#)

- [2.1 HTML](#)
- [2.2 CSS](#)
- [2.3 JavaScript](#)

[3. 8266实战](#)

- [3.1 8266的WiFi使用](#)
 - [3.1.1 STA模式\(用8266连接网络\)](#)

- [3.1.2 AP模式\(用8266创建网络\)](#)
- [3.1.3 AP+STA模式\(局域网和互联网的融合\)](#)
- [3.2 8266连接WiFi获取信息](#)
 - [3.2.1 JSON基础](#)
 - [3.2.2 使用8266获取天气信息](#)
- [3.3 AP模式挂载网页](#)
- [3.4 STA模式挂载网页](#)
- [3.5 more dynamics——WebSocket的引入](#)
 - [3.5.1 HTTP的不足——单向性](#)
 - [3.5.2 全双工协议——WebSocket](#)

附录：

- [A. 一次不怎么成功的实验结果——尝试通过软件让ESP8266实现“内网穿透-like”效果](#)
- [B. AP+STA模式代码](#)
- [C. 状态码详细介绍](#)
- [D. WebSocket Demo](#)

0. 开始之前

这里非常非常推荐对网络通信知识和前端（HTML&CSS）甚至后端比较感兴趣的同学看清华大学电子系学生科协软件部同学们的[暑培/寒培视频](#)，这一块他们更加专业。

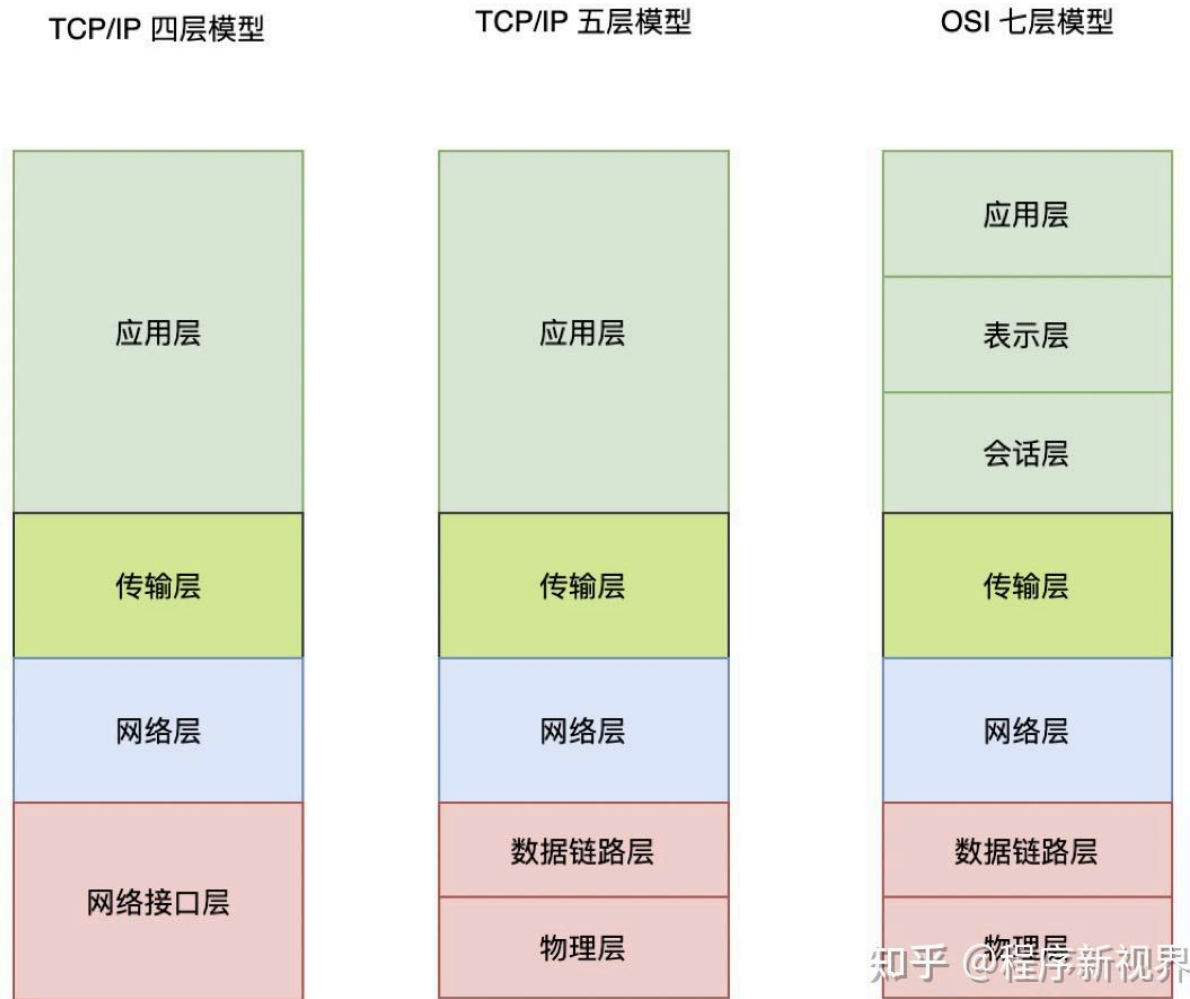
本讲义中，我主要从硬设开发的角度出发，介绍如何使用ESP8266的WiFi or 蓝牙模块实现简单的IoT功能，让大家的作品可以更出彩。在此讲义的补充材料中，JWT同学则会给大家带来如何使用树莓派部署属于自己的网页的教程（敬请期待）。

因为ESP8266本身不支持蓝牙，所以本讲不会重点涉及蓝牙的使用，会将ESP32蓝牙通讯放在附录中介绍。

#TODO 最近实在写不动了x，这个蓝牙模块后面有机会再补全吧

1. 网络基本知识

相信大家都听了N遍分层思想（比如“经典之作”——鱼骨图），网络也有比较复杂的分层设计思想，最经典的是OSI七层模型（Open Systems Interconnection Model），这里简单放一张图



对应上图的每一层我们都设计了相应的通信协议，下面是比较常见的协议：

- 会话层：RPC、SQL等
- 应用层：**HTTP**、**HTTPS**、DHCP、FTP、NFS、WAIS、TFTP、SMTP、DNS等
- 传输层：**TCP**、**UDP**等
- 网络层：**IP**、ICMP、ARP、RARP、AKP等
- 数据链路层：**以太网**、帧中继、PPP、IEEE802.3等
- 物理层：RS-232、RS-449、V.25、X.21等

这里我们不会dive into too deep，只会浅尝辄止HTTP(S)、TCP、UDP、IP、以太网这些重要概念。

为了更好的理解，我们会从下到上的介绍。

1.1 以太网(链路层)

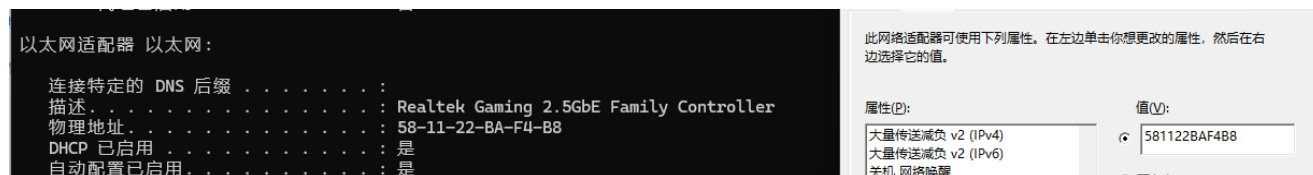
数据链路层负责在相邻节点之间无差错地传输数据帧。它在物理层传输的比特流的基础上，添加帧头和帧尾，形成数据帧，并进行差错检测和纠正。

- 以太网 (Ethernet)：一种广泛使用的局域网技术，用于在局域网内传输数据帧。
以太网使用MAC地址 (Media Access Control Address) 来唯一标识网络中的设备。每个以太网设备都有一个唯一的**MAC地址** (不是mac地址x，冷笑话😏)，用于在**局域网内进行通信**。

现在每个设备也都有一个唯一的MAC地址（MAC地址有**48位二进制数**，通常用12个十六进制数表示，如 00:1A:2B:3C:4D:5E），每个设备（准确来说是每个网卡）在出厂时都会被分配一个唯一的MAC地址。 $(2^{48} = 2.8 \times 10^{14})$ 所以还是很多的，在可预见的未来应该不会出现分配冲突问题。

PS：关于MAC地址是否可改的问题

网上的教程显示可以修改MAC地址，我实际尝试了一下确实可以，不过也有人指出只有有线网卡才能进行修改，这一点我就不去验证了。



此外还有一点很有意思，清华的校园有线网是部分是根据MAC地址验证的，你修改了自己的MAC地址就会需要重新登录，但是如果重新修改回去，那么可以直接重连而不需要重新登录。

至于为什么每次关机重启之后都需要重新登录，这个我不是很清楚（其实刚才说的根据部分根据MAC地址进行验证也是根据现象猜的，如果有对此比较了解的同学欢迎给我发邮件or微信 pat me）

#时间警察 上面是最开始的想法，后面搜了一些资料，应该是DHCP的机制，大家可以跳过上面的nonsense

#TODO 应该是DHCP的事情（待继续调研）

1.2 IP(网络层)

网络层负责数据包的路由和转发，实现不同网络之间的通信。它在数据链路层传输的数据帧的基础上，添加网络层头部，形成数据包，并根据目标IP地址进行路由选择。

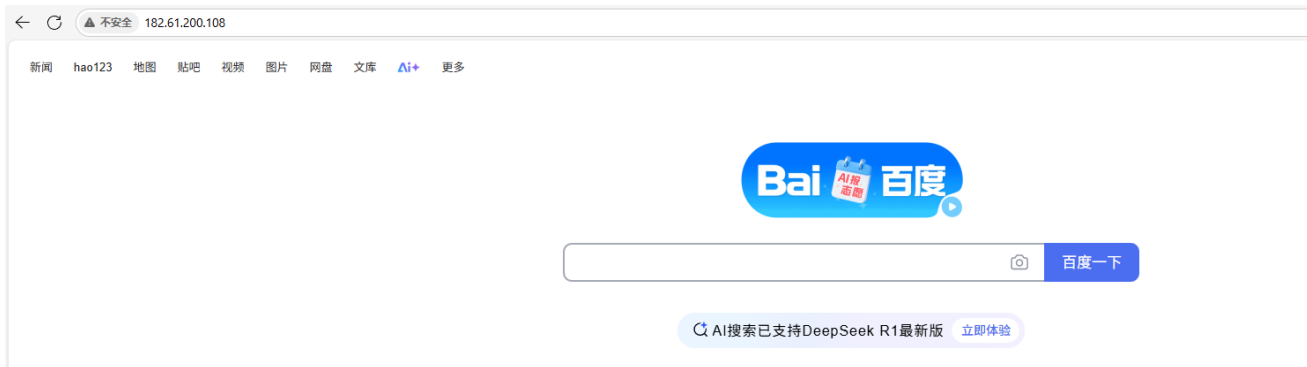
- IP (Internet Protocol)：互联网协议，负责在网络中传输数据包。IP地址是网络层的地址，用于唯一标识网络中的设备。
IP包含了IPv4和IPv6两种版本。IPv4使用32位地址空间 ($2^{32} = 4.3 \times 10^9$)，而IPv6使用128位地址空间 ($2^{128} = 3.4 \times 10^{38}$)，可以提供更大的地址空间。
目前主流的还是IPv4，IPv6也在逐步推广。

1.2.1 公网地址和私网地址

在IPv4中，IP地址是一个32位的数字(4个字节)，通常表示为四个十进制数（例如，192.168.1.1），每个数的范围是0到255（一个字节的范围）

IP地址分为公网地址和私网地址，公网地址是唯一的，可以在互联网上进行通信；私网地址是局域网内使用的地址，必须要通过NAT（Network Address Translation，网络地址转换）才能访问互联网。

公网地址是全球唯一的，比如江玮陶同学的个人主页网址：，同学们可以直接使用这个IP地址访问他的个人主页（doge）。又比如说百度的其中一个公网地址: 182.61.200.108，同学们可以直接在浏览器中输入这个IP地址访问百度的主页。（如下图）



#HOOK 注意这里显示不安全，和后面的HTTPS有关，暂时不讨论。

公网地址的分配是由互联网号码分配机构（IANA）分配给各个区域性互联网注册机构（RIR），然后由它们分配给互联网服务提供商（ISP），最后由ISP分配给最终用户。

私网地址是仅在局域网（LAN）内部使用的IP地址，它们不会被路由到互联网上。这意味着私网地址的设备不能直接从互联网接收数据包，也不能直接向互联网发送数据包。私网地址主要是为了节省公网地址资源，因为公网地址是有限的。通过使用私网地址，可以在局域网内部使用大量的IP地址，而不需要为每个设备分配公网地址。

NAT（网络地址转换）：为了使私网地址的设备能够访问互联网，通常使用NAT技术。NAT是一种网络技术，它允许多个设备共享一个公网地址。当私网地址的设备发送数据包到互联网时，NAT设备（如路由器）会将私网地址转换为公网地址，并将数据包转发到互联网。当互联网的响应数据包返回时，NAT设备会将公网地址转换回原始的私网地址，并将数据包转发给正确的设备。

1.2.2 端口(Port)

虽然每个计算机可以有一个唯一的IP地址，但是现在大家的计算机中肯定不可能只有一个程序需要和外界通信，而且不同程序之间的数据也不应该互相可见。（比如大家也不希望自己打的联网游戏还能“偷”自己邮箱的消息x）

因此IP地址解析之后还有一步需要进行，那就行进一步解析端口号，才能最终决定收到的数据究竟要发送给计算机中哪一个正在运行的程序。

端口号是一个无符号16位整数（0-65535），用于标识计算机上运行的特定程序或服务。不同的程序/服务可能会有默认的端口号要求，比如HTTP服务器程序默认端口号是80，HTTPS服务器程序默认端口号是443，这些我们后面会涉及到。

1.3 TCP和UDP(传输层)

传输层负责端到端的通信，提供可靠的数据传输服务。它在网络层传输的数据包的基础上，添加传输层头部，形成段（TCP）或数据报（UDP），并提供流量控制、差错检测和纠正等机制。

- TCP（Transmission Control Protocol）：传输控制协议，提供可靠的、面向连接的通信服务。它通过三次握手建立连接，保证数据的可靠传输。
- UDP（User Datagram Protocol）：用户数据报协议，提供不可靠的、无连接的通信服务。它适用于对实时性要求较高的应用，如视频流、语音通话等。

1.3.1 UDP简介

UDP（用户数据报协议）是一种无连接的传输层协议，主要用于需要快速传输数据但不需要可靠性的应用场景。UDP的特点包括：

- 无连接：UDP不需要在发送数据之前建立连接，也不需要再发送数据之后断开连接。
- 不可靠：UDP不保证数据包的到达顺序，也不保证数据包的完整性。发送的数据包可能会丢失、重复或乱序到达。
- 低延迟：由于UDP不需要建立连接和确认数据包的到达，它的传输延迟较低，适用于对实时性要求较高的应用，如视频流、语音通话等。

1.3.2 TCP简介

TCP（传输控制协议）是一种面向连接的传输层协议，主要用于需要可靠性和顺序性的应用场景。TCP的特点包括：

- 面向连接：TCP在发送数据之前需要建立连接，并在发送数据之后断开连接。
- 可靠：TCP保证数据包的到达顺序，并提供差错检测和纠正机制，确保数据的完整性。
- 流量控制：TCP使用滑动窗口机制来控制数据的发送速率，防止网络拥塞。
- 拥塞控制：TCP使用慢启动、拥塞避免、快速重传和快速恢复等算法来控制网络拥塞，确保网络的稳定性。

1.4 HTTP(S)(应用层)

#TODO 理论上我们应该从最基本的AT命令集开始介绍，但是因为我感觉大家暂时不太需要这些，而且我这部分也从来没有接触过，因此这版讲义我将其列为一个TODO，期待未来能有机会由我或者别的同学将其补全。

言归正传，

应用层为应用程序提供网络服务，定义了客户端和服务端之间请求和响应的格式，使得不同的应用程序能够通过网络进行通信。

HTTPS是HTTP的安全版本，所以先介绍HTTP。HTTP是Hypertext Transfer Protocol（超文本传输协议），用于在**本地Web浏览器**和**Web服务器**之间传输数据。HTTP使用TCP作为传输层协议（为了保证**稳定性**），通常使用80端口进行通信。

HTTP是**万维网(www)** 数据通信的基础。

超文本是什么？简单来说，超文本（Hypertext）是一种文本形式，它包含有可以链接到其他资源（如其他文本、图片、视频等）的超链接。超文本（Hypertext）是一种文本形式，它包含有可以链接到其他资源（如其他文本、图片、视频等）的超链接。

这个描述是否让你想到一些什么？没错，超文本就是我们常用的网页内容，HTML（Hypertext Markup Language）就是用来描述超文本的标记语言。HTML结合上CSS（Cascading Style Sheets，层叠样式表）以及JavaScript内容，可以在**本地浏览器**上渲染出丰富的网页内容。

Notes 🌈：所有的网页其实都是本地浏览器接受一些文件在本地渲染出来的，这一点后面会更详细的介绍，这里让大家有个印象。

1.4.1 HTTP(超文本传输协议)

HTTP是一个经典的请求-响应协议 (server-client model)，客户端（通常是浏览器）向服务器发送请求，服务器处理请求并返回响应。

具体流程如下：

1. 客户端和服务端建立连接：具体来说，如果我们使用的是**域名**(比如baidu.com)，那就要首先**使用DNS解析域名**得到服务器的IP地址(对于大型网站来说，可能会有多个IP地址)，然后使用TCP建立连接，这个建立链接的过程我们不做解读。
2. 客户端向服务器发出请求：一旦建立了TCP连接，客户端就会向服务器发送HTTP请求（后面详细解读HTTP请求的格式）
3. 服务器收到请求，返回相应的文件作为响应 (Response)：在协议刚开始，传输的一般都是超文本HTML文件，但是随着网络的发展，HTTP协议也支持传输其他类型的文件，如图片、视频、音频等。
4. 客户端和服务端关闭连接：这一步是为了保证双方正常关闭连接，释放网络资源（这一部分我们也用不到，所以不做解读）

什么是DNS？前面我们提到了DNS，这是一个新的概念，因此下一部分我们介绍：

1.4.2 当我们使用域名的时候，背后发生了什么？

当你在浏览器中自然的输入www.baidu.com的时候，你可能并没有意识到背后发生了什么...

当你在《爱情公寓》中看到曾小贤（应该是他？）举着“根服务器”(剧中所称的“互联网的本体”)的时候，你可能还没有意识到今天你要和它再打交道（有人还记得这个桥段吗x，这里不

讨论这个情节是否涉及抄袭的问题(叠甲))



你应该还记得上上节我们刚介绍完IP地址的概念，每个网站都有至少一个公网IP，每次访问网站本质上也就是访问一个公网IP。

那么，我们是怎么从一个 www.baidu.com 的字符串转换到 182.61.200.108 这一串数字的呢？

这就需要DNS了，DNS(Domain Name System,域名系统)是当代互联网的一个核心服务，它把我们从复杂无规律的32位IP地址（对于IPv4来说）背诵中解脱出来，我们只需要记住朗朗上口的www.google.com，www.baidu.com e.t.c就行

1.4.2.1 如何在《地图》上找到一个域名

就像在地图上确定一个旅游地点：日本--关东地区--东京--东京塔一样，域名也是具有有趣的分级和从属结构，我们以www.tsinghua.edu.cn为例，域名的解析是从右向左的：

- 最右边 .cn 这是一个**顶级域名**，表示这个域名属于中国，顶级域名有几个典型的类型：一个是这种**国家**，比如 .cn ， .us ，另一个就是 .com ， .org 这些表示**通用类别**（.com 一

般是公司，.org 一般是各种非营利组织、国际机构等，但是这么说其实也不全对，因为**很多域名都是可以个人租的**，这里大家可以自行Google一下（，这里的生意还是挺有意思的x，大家感兴趣也可以尝试租一个不贵的、觉得喜欢的域名，部署自己的网页之类的x

- .edu 这是一个**二级域名**，表示这是一个教育机构。当然有同学可能很好奇，像是MIT这些的域名是 mit.edu，为什么国内大学普遍是 .edu.cn 呢？这就是一个作用域问题（梦回程设），.edu.cn 是我国在顶级域名 .cn 下单独设置的二级域名，专门给大学机构使用，而 .edu 顶级域名是在美国的VeriSign公司手中（GPT生成未验证x），需要向其租用，非常不方便
- .tsinghua 则是**三级域名**，在 .edu.cn 中具体区分这是清华大学的域名
- www 是**子域名**，这里面非常有意思，www. 基本上就表示网站的主服务器的域名。关于目前大家只输入 baidu.com 也会指向baidu的主页面，这是因为没有指定子域名的话，DNS 会**默认解析出子域名 www.**

mails.tsinghua.edu.cn 也是类似分析，其他所有网站也都遵循相同的准则。

1.4.2.2 How about the Machines?

现在大家可以理解域名是**如何组成**的了，但是机器如何获得域名对应的IP地址呢？给每个机器出厂的时候写进去是不现实的，第一因为IP地址对应的域名是会**变化**的（租用or退租，甚至某些公司倒闭e.t.c），第二这些数据写进来会占据相当大的内存。所以实际上是随用随查的，查询的过程就是DNS参与进来的过程。

假设我们第一次访问 www.tsinghua.edu.cn，那么过程是这样的：首先浏览器查看自己的缓存，发现没有访问过，不知道这是个什么网站；

OK，再查询本机操作系统的缓存（比如Windows系统中的DNS Client Service相关文件），还是找不到；

然后向**本地配置的DNS服务器**进行**递归查询**，如果本地配置的DNS服务器有缓存记录，就会返回。（实际上大家查看自己的本地DNS服务器配置，会发现非常好玩：我的电脑显示是这样的

```
DNS 服务器 . . . . . : 2402:f000:1:801::8:28
                    2402:f000:1:801::8:29
                    166.111.8.28
                    166.111.8.29
                    101.7.8.9
```

根据网上的[Blog](#)显示在IPv4环境下，清华的DNS服务器地址主要有166.111.8.28和166.111.8.29等；在IPv6环境下，其DNS地址为2001:da8:200:ffff::28 不过我觉得这里的IPv6的29也完全可以**合理推测**为清华的DNS服务器在IPv6下的地址。

那么剩下的101.7.8.9又是哪个服务器呢？使用网上到处都是的**IP查询工具**，我们查到这个是北京教务网的DNS服务器。

请输入IP或网站域名:

IP 地址:	101.7.8.9
IP Long:	1694959625

高精归属地定位 (IP数据云)

中国 上海 上海

归属地(纯真数据):	中国 北京 北京 教育网/DNS服务器
归属地(ipip):	中国 北京 北京 -
归属地(淘宝数据):	中国 湖北 武汉 教育网
归属地(IP2REGION):	中国 北京 北京市 教育网
归属地(GeoLite2):	China -
归属地(DbIp):	China -

显然上述的服务器绝对会有我们访问的网站的记录，从而一定记录了其IP地址，但是，我们不妨假设这些服务器全被“黑”了，这个域名的记录全被删掉了，那么我们该如何查询呢？

作为递归查询，本地配置的几个DNS服务器找不到，就会真正根据这个域名的组成从顶级域名开始查起，不断跳转DNS服务器（从 .cn 服务器到 .edu 服务器....）一直查到子域名 www. 才会最后给出这个域名对应的IP地址（实际上是一个IPv6地址：

2402:f000:1:402:101:6:15:66)

(但是很遗憾，这个域名是无法直接访问的，我觉得应该是清华为了防止被网页攻击做的一些防御措施，目前知识水平不够，无法理解)

对于已经访问过的，浏览器一般会保留有缓存，从而不需要再这么麻烦的不断跳转的访问，这也是一种通过缓存数据了来加速访问速度的办法（数逻老师狂喜）

PS：还有一个好玩的，你可以通过写操作系统的DNS缓存，从而把一些域名解析到你指定的IP地址中，从而能实现一些简单的便捷功能，也能实现一些粗浅的攻击，但是这一块就不做过多描述了。

1.4.3 HTTP协议如何实现通信的？

1.4.3.1 HTTP请求格式

最经典的HTTP的每次请求是一次性的，客户端向服务器发送一个HTTP请求，可能是请求某些文件，也可能是请求修改服务器中的某些数据等等。然后服务器返回一个响应，链接就可以断开了。

而在这一一次性的请求中，HTTP格式规定要包含如下几个部分：

```
<请求行>---<请求头>---<空行>---<请求体>
```

以GET请求为例：

```
GET /index.html HTTP/1.1
Host: www.example.com
```

User-Agent: Mozilla/5.0**Accept:** text/html

- 请求行（包含了**方法**、**请求路径**、**协议版本**）
 - 方法：表示请求的类型，常见的有：GET（获取资源）、POST（提交数据）、PUT（更新资源）、DELETE（删除资源）等。
 - 请求路径：表示要访问的资源路径，比如 `/index.html`。表示想要请求的资源是在服务器上的相对位置（相对于服务器的根目录）（下面的表给出了一些常见的请求路径的含义，部分由G老师提供，未完全验证正确性）
 - 协议版本：一般都使用 HTTP/1.1（算是比较新的HTTP协议版本）
- 请求头（包含了**请求的元信息**，一般都是一系列**键值对**）
 - Host：表示请求的主机名，通常是域名或IP地址。
 - User-Agent：表示**客户端**的浏览器信息。
 - Accept：表示客户端能够处理的内容类型。
 - 下面有表格进一步介绍了请求头的其他字段
- 空行（一个没有任何实质内容的行，可能会有 `\r\n` 这种内容，用于表示请求头的结束）
- 请求体（可选，通常用于POST、PUT等请求，用于在请求中**携带数据**，发送给服务器，GET一般没有请求体）
 - 请求体的内容是什么格式主要取决于前面请求头中 Content-Type 字段的值，比如 `application/json` 表示请求体是JSON格式，`application/x-www-form-urlencoded` 表示请求体是表单数据格式。

比如：

请求头：Content-Type: application/x-www-form-urlencoded

请求体：username=jack&password=123456

// or

请求头：Content-Type: application/json

请求体：(JSON-type Data)

```
{
  "username": "jack",
  "password": "123456"
}
```

示例路径	含义说明
<code>/</code>	网站首页（根目录）
<code>/index.html</code>	请求 index.html 静态网页
<code>/css/style.css</code>	请求 CSS 资源
<code>/api/user</code>	RESTful API 请求用户信息
<code>/search?q=esp8266</code>	携带查询参数（query string）

请求头字段	说明
Accept	指定客户端能接收的数据类型。例如：Accept: text/html, application/json
Accept-Encoding	指定客户端支持的压缩方式，如：gzip, deflate
Accept-Language	指定客户端语言偏好。例如：Accept-Language: zh-CN, en-US
Connection	控制连接方式：keep-alive（长连接）或 close（请求完就断）
Content-Type	告知服务器请求体的数据格式（主要用于 POST/PUT）。例如：application/json, application/x-www-form-urlencoded
Content-Length	请求体（Body）的长度（字节数）
Authorization	传递身份验证信息（如Bearer Token、Basic Auth）
Cookie	向服务器发送保存在客户端的 cookies，用于会话维持
Referer	告诉服务器用户是从哪个页面跳转来的，常用于统计或防盗链
Origin	跨域请求时，表示请求的源（协议 + 域名 + 端口），用于 CORS 安全控制

1.4.3.2 HTTP响应格式

有了请求，服务器收到请求之后还需要返回一个响应（无论是否请求是否成功）然后这次连接才能完成。而HTTP协议规定的响应格式如下：

```
<状态行>---<响应头>---<空行>---<响应体>
```

以一个非常简单的成功响应为例：

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 88
Connection: keep-alive
Date: Wed, 26 Jun 2025 08:00:00 GMT

<html>
  <head><title>Example</title></head>
  <body><h1>Hello, world!</h1></body>
</html>
```

- 状态行（包含了**协议版本**、**状态码**、**状态描述**）
 - 协议版本：一般都是 HTTP/1.1。
 - 状态码：表示请求的处理结果，常见的有：

- 200 OK：请求成功。
- 404 Not Found：请求的资源不存在。
- 500 Internal Server Error：服务器内部错误。
- 附录C. [状态码详细介绍](#)有更详细的介绍状态码对应的含义
- 状态描述：状态码的英文描述，主要用于人类阅读，不用于程序判断。可根据服务器不同略有差异。
- 响应头（包含了响应的元信息，和请求头类似，也是一系列键值对）
 - **Content-Type**：表示响应体的内容类型，比如 text/html 表示HTML内容，text/plain 表示普通文本，application/json 表示JSON数据。（因为响应一般是有各种文件在响应体中的，所以 Content-Type 是很重要的）此外，还能顺便指定编码格式
 - Content-Length：表示响应体的长度（字节数）。
 - Connection：表示连接状态，通常是 keep-alive（长连接）或 close（短连接）。（这里是和之前HTTP请求一次就断开不太一样的点）
 - Date：表示响应的日期和时间。
- 空行（同上请求部分）
- 响应体（如果找到了请求的资源，服务器就会返回给客户端一些文件资源）
 - 响应体的内容可以是HTML、JSON、图片、视频等各种类型的文件，具体取决于请求的资源类型。响应体的解码方式**直接由响应头的 Content-Type 字段决定**。（上面的例子给的就是传输一个非常简单的HTML文件，并且也指定了浏览器要用HTML格式来渲染这个文件

1.4.3.3 8266实例

因为内容非常多，我们后面单独使用[3. 8266实战](#)来介绍。

1.4.4 这个S到底意味着什么？带来了哪些变化？

“HTTPS”中的“S”代表“Secure”（安全），表示该协议在传统HTTP的基础上增加了安全保护机制。HTTPS即HTTP over TLS/SSL，是通过在HTTP协议上叠加传输层安全协议（TLS，前身为SSL）实现的数据加密传输方式。其核心目的是保障客户端与服务器之间通信的机密性、完整性以及身份真实性。

具体来说，HTTPS相较于HTTP带来了以下关键变化：

1. 数据加密传输

HTTPS利用TLS/SSL协议对传输的数据进行加密处理，确保通信内容在传输过程中不会被第三方窃听或篡改，极大提升了数据的安全性。

2. 身份认证

服务器需持有由受信任的证书颁发机构（CA）签发的数字证书，客户端在连接时会验证该证书，从而确认服务器身份，防止中间人攻击（MITM）。

3. 数据完整性保证

通过消息认证码（MAC）和加密校验，保证数据在传输过程中未被非法修改，确保数据的

完整性。

4. 增强用户信任感

现代浏览器通过锁形图标及安全警告提示用户网站是否采用HTTPS，提升用户访问网站的安全感，特别是在涉及用户隐私、账号登录及在线支付等敏感操作时。

5. 端口及协议支持

HTTPS默认使用443端口，与HTTP的80端口区分，同时支持包括HTTP/2在内的现代传输协议，以提高传输效率。

（显然上述文字是G老师写的，我写到现在其实**一直都没用HTTPS进行连接**，这确实是我整个讲义中还存在的问题，希望未来能进行改正）

#TODO 添加HTTPS连接实例讲解

2. HTML & CSS & JavaScript

本节看起来内容会很多，但是实际上我们只需要了解其基本的概念和语法即可，毕竟现在GPT生成的网页代码已经非常成熟了x，自己再从头开始设计前端并非一个非常明智的选择。

2.1 HTML

HTML，前面提到是Hypertext Markup Language（超文本标记语言），是用来创建网页的标准标记语言。具体来说，HTML使用标签（Tags）来定义网页的结构和内容。一个HTML文档通常包含下面几个部分：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>网页标题</title>
</head>
<body>
  <h1>这是一个标题</h1>
  <p>这是一个段落。</p>
</body>
</html>
```

大家可以在比如Vscode中创建一个 .html 文件，然后将上述代码复制进去，用浏览器打开这个文件，就能看到浏览器根据这段HTML代码渲染出来的网页内容了。（下图是有修改的代码

渲染出来的结果)



- `<!DOCTYPE html>` 声明文档类型，告诉浏览器这是一个HTML5文档。(HTML5就是现代版HTML标准)
- `<html>` 标签表示HTML文档的开始，`lang="en"` 表示文档的语言是英语（可以根据需要修改为其他语言，如 `zh-CN` 表示简体中文）。
- `<head>` 标签包含文档的**元数据**（如标题、字符集等）：
`charset=UTF-8` 表示文档使用**UTF-8**编码（大家应该或多或少在程设课上听说了编码这一概念？这里不重要，就不详细介绍了，简单来说：UTF-8是一种广泛使用的字符编码，它支持几乎所有语言的字符，包括中文、英文、日文等。）
`<meta name="viewport" content="width=device-width, initial-scale=1.0">` 表示文档在设备上的显示效果，`width=device-width` 表示显示**宽度为设备宽度**，`initial-scale=1.0` 表示初始缩放比例为1.0。这是一步非常重要的设置，因为这决定了在不同的设备上打开网页，显示的宽度可以**自适应**不同的屏幕大小。
`head` 的 `title` 则决定打开的**网页标题**，比如我改成 `网页标题1111`，那么实际效果就有上图红色框标注的部分显示的 `网页标题1111`。
- `<body>` 标签内则是文档的**主体内容**：
`<h1>` 标签表示一个一级标题，相应的还有 `<h2>`（二级标题）、`<h3>`（三级标题）等，数字越小标题级别越高，字体也越大。可以一直写到 `<h6>`。

`<p>` 标签表示一个段落，相应的还有 `<div>` 标签用于定义一个块级元素，可以包含其他HTML元素。（`<div>` 标签非常重要，因为它可以用来组织和布局网页内容，通常用于将相关的内容分组在一起）

`<a>` 标签用于创建超链接，可以链接到其他网页或资源。基本的使用方式如下：`<a href="https://www.google.com" target="_blank">Google Link</a>`，其中 `href` 属性指定链接的目标地址，`target="_blank"` 表示在新窗口打开链接。

`<img>` 标签用于插入图片，可以使用 `src` 属性指定图片的路径，如 ``，其中 `alt` 属性用于提供图片的替代文本（当图片无法加载时显示）。**注意** ⚠️，后面我们一般不会使用本地的图片，因为8266的内存相当小。而是使用在线图片的链接，比如下面：

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>示例网页</title>
</head>

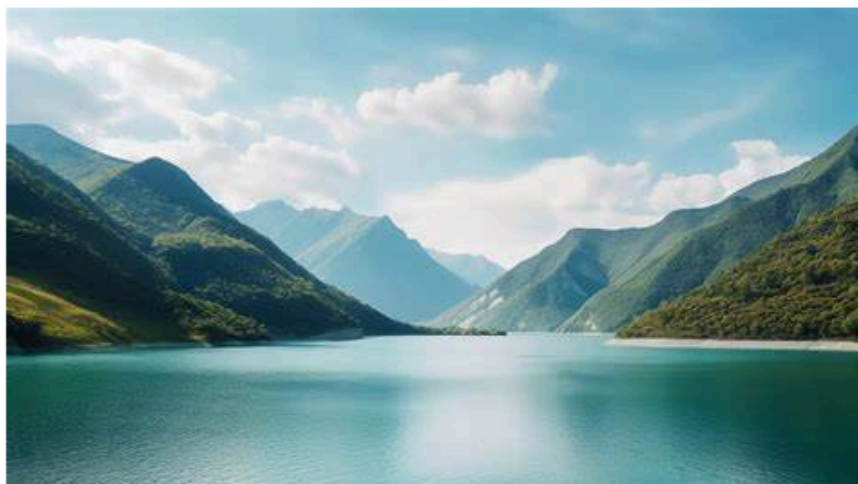
<body>
  <h1>欢迎来到我的网页</h1>
  <p>这是一个示例网页，展示如何插入网络图片。</p>
  
  <p>以上是一张来自网络的示例图片。</p>
</body>

```

浏览器渲染出来的效果如下：

欢迎来到我的网页

这是一个示例网页，展示如何插入网络图片。



以上是一张来自网络的示例图片。

建议： 在使用的时候稍微注意一下图片的版权问题x，虽然很多网站自己都是盗转的就是了x

其他的很多特化的很有用的HTML标签（比如 <table>），大家有需要可以上网自行google一下，或者问问G老师。

2.2 CSS

CSS（Cascading Style Sheets，层叠样式表）是一种用于描述HTML文档外观和格式的样式表语言。它允许我们控制网页的布局、颜色、字体、间距等视觉效果。

一般的格式就是声明一个HTML中存在的标签名称，然后在大括号中指明其样式属性和值，比如：

```
h1 {
    color: blue; /* 设置颜色为蓝色 */
    font-size: 24px; /* 设置字体大小为24像素 */
}
```

但是有时候我们不想要同一个标签有完全一样的样式，那么我们需要在HTML中给标签加上 `class=xxx` 或者 `id=xxx` 的属性，并在CSS中使用 `.className`（类选择器）或者 `#idName`（ID选择器）来定义样式。

比如：

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>CSS示例</title>
    <style>
        .red-text {
            color: red; /* 设置颜色为红色 */
        }
        #unique-heading {
            font-size: 30px; /* 设置字体大小为30像素 */
        }
    </style>
</head>
<body>
    <h1 id="unique-heading">这是一个独特的标题</h1>
    <p class="red-text">这是一个红色的段落。</p>
</body>
```

注意 ⚠： CSS设置会有覆盖情况，后面的样式设置会覆盖前面的设置，如果网页CSS效果不是自己想要的，可以检查一下是否有覆盖的情况。

CSS可以通过三种方式应用到HTML文档中：

1. 内联样式（Inline Styles）

内联样式是直接在HTML元素中使用**style 属性**来定义样式。这种方式的优先级最高，但不推荐广泛使用，因为它会破坏HTML的语义结构，使HTML和CSS代码混合在一起。比如 `<h1 style="color: blue; font-size: 24px;">欢迎来到我的网页</h1>`，渲染结果就是蓝色，24px大小的一级标题

2. 内部样式表（Internal Styles）

内部样式表是将CSS代码放在HTML文档的 `<head>` 部分的 `<style>` 标签中。这种方式适

用于单个HTML文档的样式定义。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>内部样式表示例</title>
  <style>
    h1 {
      color: blue;
      font-size: 24px;
    }
    p {
      color: green;
      font-family: Arial, sans-serif;
    }
  </style>
</head>
<body>
  <h1>欢迎来到我的网页</h1>
  <p>这是一个示例网页，展示如何使用内部样式表。</p>
</body>
</html>
```

3. 外部样式表 (External Styles)

外部样式表是将CSS代码保存在一个单独的 `.css` 文件中，然后通过HTML文档中的 `<link>` 标签引入。这种方式适用于多个HTML文档共享相同的样式定义，便于维护和管理。比如下面：

```
h1 {
  color: blue;
  font-size: 24px;
}
p {
  color: green;
  font-family: Arial, sans-serif;
}
```

然后在HTML文档中引入这个CSS文件：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>外部样式表示例</title>
```

```

<link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>欢迎来到我的网页</h1>
  <p>这是一个示例网页，展示如何使用外部样式表。</p>
</body>
</html>

```

(如上链接外部css文件是最常用的用法，注意这里的相对路径，表示style.css和demo.html文件应该是在同一个文件夹下的)(其他 <link> 标签的使用方法大家可以问问G老师，这里不做过多介绍)

如果你的CSS文件需要在多个网页中使用，推荐使用外部样式表。但是对于我们8266的使用实践来说，**一般也就一个主网页**，所以我们一般更习惯直接在HTML文件中使用内部样式表(除非你的**CSS设计非常复杂**，或者你是从网上找的一些模版代码，这个时候可以考虑使用外部样式表)。

2.3 JavaScript

一般来说，JavaScript的引入给网页带来了动态交互的能力，使得网页不仅仅是静态的文本和图片，而是可以响应用户的操作，进行数据处理和展示。

JavaScript是一个非常强大的语言，但是作为一个编程语言，即使只是讲解基础语法，也会非常大的增加内容容量，这里就不赘述，感兴趣同学参考[JavaScript 教程 | 菜鸟教程](#) or/and 软件部寒培材料即可，里面讲的还是很详尽的。

3. 8266实战

3.1 8266的WiFi使用

这一部分还是基础知识。

8266支持三种WiFi工作模式，分别是STA (Station Model)、AP (Access Point) 和 AP+STA (混合模式)

我们这里无意去探究各个工作模式的底层原理，我们更侧重如何使用。因此后面我会**尽可能把代码都给出来**但是不会给实际的演示视频，主要很希望大家能自己动手试一下，**动手尝试**才能发现其中可能自己没完全理解的地方。

3.1.1 STA模式(用8266连接网络)

Station模式是8266作为一个**客户端**连接到**现有的WiFi网络**中。它可以连接到一个路由器或者说是接入点 (AP)，并通过这个网络访问互联网(如果有NAT服务)或其他设备。

比如我们开启手机热点，设置好网络名称(也就是SSID)和密码，然后就可以编程让8266连接到这个网络中。类似的，对于一些登录比较简单的WiFi网络，在知道网络名称和密码的条件下也可以快速连接。

注意 ⚠： 热点**不能设置为5GHz频段** (华为手机热点默认开启就是5GHz频段，其他手机应该也都是，需要各位同学注意一下这点)，**需要设置为2.4GHz频段**，否则8266是接收不到信号

的。



注意 ⚠： 这里还要注意有些手机的热点是可以同时在连接WiFi的时候开启，此时原理应该是做一个信号的转发？反正结果就是这个时候我们是**无法在共享设置中找到AP频段设置选项**的（应该是默认沿用目前连接WiFi的频段，而清华的网络一般是5G频段的，因为我试了ESP8266这个时候是无法找到网络的），所以如果你发现手机热点设置中**没有AP频段选项**，那么就需要先**断开WiFi连接，再开启手机热点**，才能看到AP频段设置选项。

具体8266连接WiFi的操作：

```
#include <ESP8266WiFi.h> // 引入ESP8266 WiFi库
// 替换为你的WiFi名称和密码
const char* ssid = "ESP8266_TEST_WIFI"; // WiFi名称
const char* password = "qasd1112222"; // WiFi密码

void setup() {
    Serial.begin(115200); // 初始化串口通信，波特率设置为115200
    delay(10);
    // 连接到WiFi网络
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);
    WiFi.begin(ssid, password); // 开始连接WiFi
```



```

while (WiFi.status() != WL_CONNECTED) { // 检查是否连接成功
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP()); // 打印分配到的IP地址
}
void loop() {
    // 在这里可以添加其他代码，例如发送数据到服务器等
    delay(1000); // 延时1秒
    Serial.println("IDLING"); // do something
}

```

首先引入 <ESP8266WiFi.h> 这个库，然后定义ssid和password常量（这一点不赘述）然后在 setup() 函数中初始化串口通信，使用 WiFi.begin(ssid, password) 开始连接WiFi，并使用 while (WiFi.status() != WL_CONNECTED) 循环等待连接成功。（可以增加超时退出逻辑，同学们自行决定）

连接成功后，使用 WiFi.localIP() 获取连接网络之后**8266被分配到的IP地址**，并打印出来。在 loop() 函数中，我们可以添加其他代码，比如发送数据到服务器等，这里我们只是**简单地打印出"IDLING"**来表示8266正在工作。

STA模式下，我们经常使用WiFiClient进行互联网访问，下面是一个简单的例子，展示如何使用8266获取洛杉矶当地时间消息，并通过串口打印出来：

```

#include <ESP8266WiFi.h>
#include <WiFiClient.h>

// 替换为你的WiFi网络名称和密码
const char* ssid = "your_SSID";
const char* password = "your_PASSWORD";

// 测试连接的网站
const char* testServer = "www.baidu.com";
const int httpPort = 80;

void setup() {
    Serial.begin(115200);

    // 连接到WiFi网络
    WiFi.begin(ssid, password);
    Serial.print("Connecting to WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConnected to WiFi");
}

```

```

// 测试连接到 baidu.com
WiFiClient client;
if (client.connect(testServer, httpPort)) {
    Serial.println("Connected to baidu.com");
    // 发送HTTP GET请求
    client.print(String("GET / HTTP/1.1\r\n") +
                  "Host: " + testServer + "\r\n" +
                  "Connection: close\r\n\r\n");
} else {
    Serial.println("Failed to connect to baidu.com");
    return;
}

// 等待服务器响应
unsigned long timeout = millis();
while (client.available() == 0) {
    if (millis() - timeout > 5000) {
        Serial.println(">>> Client Timeout!");
        client.stop();
        return;
    }
}

// 读取并打印响应
String line;
while (client.available()) {
    line = client.readStringUntil('\n');
    Serial.println(line);
}

client.stop();
}

void loop() {
    // 无操作，测试在setup中完成
}

```

值得注意的点有：

- WiFiClient需要**手动设置**HTTP请求格式（请求行、请求头等）
- connect()函数用于验证能否连接，相当于我们电脑终端常用的 ping 命令
- client.available()用于检查接受数据的缓冲区是否还有数据可读（类比于程设课程里的内容，这里实现的功能就是读取一直到换行符 \n，然后把整行的内容输出出来（println 自带换行符）
- client.stop() 函数用于关闭连接，释放资源。
输出结果就是请求回来一个baidu.com的HTML文件（

原本我希望展示一个获取洛杉矶当地时间的例子，但是由于洛杉矶时间需要使用API来获取，再加上有些网站连接一言难尽，所以这里就退而求其次了，主要是希望大家知道我们的程序在干什么。

3.1.2 AP模式(用8266创建网络)

上面可以看到频段名称为：**AP频段**。8266也有AP模式，在这种模式下，8266可以作为一个接入点（Access Point），**创建一个自己的WiFi网络**。其他设备可以连接到这个网络中，从而与8266进行通信。（此时是一个局域网，无法连接互联网）

注意 ⚠： 不要不改ssid和password的值的就去烧录代码，不然你可能会看到**十几个**名为 ESP8266_AP 的WiFi网络出现在你的手机或电脑的WiFi列表中（**假如你的周围同学正好都在做这个实验**）。所以SSID一定要**改成自己能分辨出来的名字**再烧录！

具体8266创建WiFi网络的操作：

```
#include <ESP8266WiFi.h> // 引入ESP8266 WiFi库
// 设置AP模式的WiFi网络名称和密码
const char* ssid = "ESP8266_AP"; // WiFi网络名称
const char* password = "12345678"; // WiFi网络密码

void setup() {
    Serial.begin(115200); // 初始化串口通信，波特率设置为115200
    delay(10);
    // 设置ESP8266为AP模式
    WiFi.softAP(ssid, password); // 创建WiFi网络
    Serial.println("AP mode started");
    Serial.print("AP IP address: ");
    Serial.println(WiFi.softAPIP()); // 打印AP模式下的IP地址
}

void loop() {
    // 获取当前连接的设备数量
    int clientCount = WiFi.softAPgetStationNum();
    Serial.print("Connected clients: ");
    Serial.println(clientCount);
    delay(3000); // 每3秒更新一次连接设备信息
}
```

#Notes 很遗憾这里ESP8266的标准WiFi库是不支持获取连接到8266的设备的IP地址的，根据网上的搜索结果ESP32是支持的，对此感兴趣的同学可以尝试使用ESP32的WiFi库实现这个功能。

根据资料显示，ESP8266的AP模式下，所有的IP地址都是分布在 192.168.4.1 到 192.168.4.254 之间的，其中 192.168.4.1 默认是8266本身的IP地址。

想验证这个同学可以使用下面的代码进行验证：

```

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
const char* ssid = "ESP8266_AP"; // WiFi网络名称
const char* password = "12345678"; // WiFi网络密码

ESP8266WebServer server(80); // 创建一个Web服务器对象，端口为80
void handleRoot() {
    String message = "Hello, this is ESP8266 AP. Your IP address is: ";
    message += server.client().remoteIP().toString(); // 获取连接设备的IP地址
    server.send(200, "text/plain", message); // 发送响应
}
void setup() {
    Serial.begin(115200);
    delay(10);
    // 设置ESP8266为AP模式
    WiFi.softAP(ssid, password);

    Serial.println("AP mode started");
    Serial.print("AP IP address: ");
    Serial.println(WiFi.softAPIP());
    // 设置Web服务器的根路径处理函数
    server.on("/", handleRoot);
    server.begin(); // 启动Web服务器
}
void loop() {
    server.handleClient(); // 处理Web服务器的客户端请求
}

```

使用方式：烧录到8266上之后连接网络，在浏览器中输入 192.168.4.1，这会访问8266的Web服务器，从而获取一个携带自身IP地址 `server.client().remoteIP().toString()` 的返回结果，一般情况下，应该是按照连接顺序分配 .2 到 .254 的IP地址（maybe？但是大概率你也找不到这么多个设备连起来，更别说这么多设备能否同时连接到8266上了x）

这里涉及到了两个设备之间的通信，因此我们可以使用HTTP协议的内容来解读这段代码：

从8266的视角分析：

1. 首先，其创建了一个服务器对象 `server`，对应的端口是80（默认HTTP端口都是80）
注意 ⚠： HTTP服务默认端口号是80，当我们输入 192.168.4.1 的时候我们并没有输入对应的端口号，这是因为这是一个HTTP请求，会自动解析到80端口，所以如果我们开的服务器监听端口确实是80，那么一切正常，但是如果设置的端口不是80，比如我们设置为监听端口81，那么还输入 192.168.4.1 就收不到任何响应，因为此时8266的server是监听不到请求的，也就自然不会发送回响应。此时我们需要输入 192.168.4.1:81，明确告知服务器这是一个解析到81端口的请求。
2. 其次，其定义好了响应请求的函数 `handleRoot()`，`message`就是一个简单的文本信息，关键在于 `send`，这里就是针对客户端的HTTP请求做出回复，因为这是在一次HTTP连接

中的，所以会自动进行，不需要指定客户端的IP地址等消息。状态码、Content-Type、响应体这些内容我们前面都讲过了

- 最后设置我们定义的函数用于响应根路径请求 `server.on("/", handleRoot)`，其实也就是设置网页路由

#Notes 上面提到了**设置网页路由**的概念，在 Web 开发中，“路由 (Route)”指的是：**某个 URL 路径（如 /、/login、/data 等）与对应处理函数（代码）之间的映射关系**。当浏览器访问某个路径时，Web 服务器会根据这个映射，调用预先定义的函数来处理该请求。

4. `setup`函数的最后启动服务器 `server.begin()`

5. `loop`函数中我们需要监听客户端是否有请求过来，这里使用 `server.handleClient()` 来处理客户端请求。这是 ESP8266WebServer 库中的一个方法，用于处理客户端的请求。当有客户端（如浏览器）连接到 ESP8266 的 Web 服务器时，`handleClient()` 方法会检查是否有新的请求到来，并根据请求的内容调用相应的处理函数。在你的代码中，当客户端访问 Web 服务器的根路径（/）时，`handleClient()` 方法会调用 `handleRoot` 函数来处理请求。

注意 ⚠：我们这里只定义了根路径下的处理函数，因此**其他路径的访问请求全都没有结果（是连状态码也没有返回的没有结果）**。

从客户端的视角分析（比如我们使用另一个手机连接WiFi）：

连接好网络之后，我们在浏览器中输入 192.168.4.1，浏览器会自动发送一个HTTP请求到8266的默认端口80，这个请求的格式可以推测是这样的：（细节可能不足，但是基本上是这样的）

```
GET / HTTP/1.1
Host: 192.168.4.1
User-Agent: Mozilla/5.0 (Linux; Android 11; Pixel 5)
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
```

然后8266会根据`handleRoot`函数返回一个响应：这个相应的状态码是200（成功），Content-Type=text/plain，意味着我们手机上的浏览器需要将响应体当做纯文本直接显示出来，响应体是message（内容见上文）。浏览器忠实的显示结果，我们看到，整个过程结束。

3.1.3 AP+STA模式(局域网和互联网的融合)

一般来说，AP+STA模式的作用主要在于8266连接路由器获取互联网信息的同时和局域网的多个设备进行沟通。比如我可以把若干个本地局域网设备的数据发送给8266，然后8266再通过互联网发送到比如云服务器上，从而实现云端来监控这些设备的数据状态。（当然连接云服务器的整个过程会复杂一些，对此感兴趣的同学可以找拓展资料了解学习）

但是，我在最开始接触AP+STA融合模式的时候，我的期待其实是我可以将本地的互联网访问通过8266--路由器这样的一个转发，实现我当时所期待的“内网传统”效果，当然这个是不现实的，因为首先ESP8266并不支持直接的网络共享（根据网上的资料显示ESP32支持，这部分

尝试留待同学们尝试将学到的这些内容**迁移应用**到ESP32上）后面我会在附录中介绍如何 使用软件去尽可能的实现类似的功能（不过效果比较差，大家看了感觉好玩就行）

这里给出AP+STA模式的代码：我们这里实现一个最简单的从局域网中收集信号（这里是从连接局域网的手机收集，然后利用Github上给出的一个[Webhook](#) 收集POST请求。

代码如下：（因为代码过长，为了保证前面正文的连贯性，我将其放在了附录中）

B. AP+STA模式代码

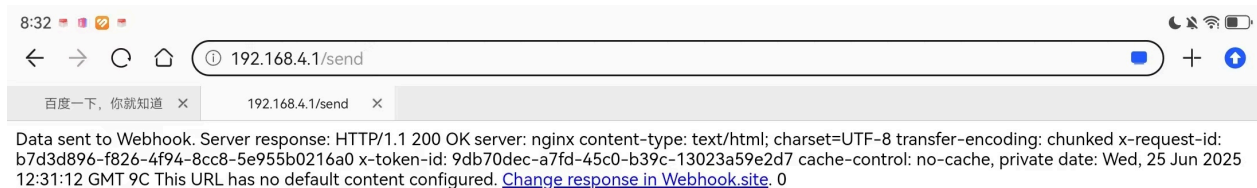
经过前面的HTTP协议的介绍，大家这里也基本上知道这段代码是如何工作的了，因此这里留给各位同学自己尝试解读一下每一段的作用，以及在HTTP协议中每部分的具体作用。

具体实现的效果如下：

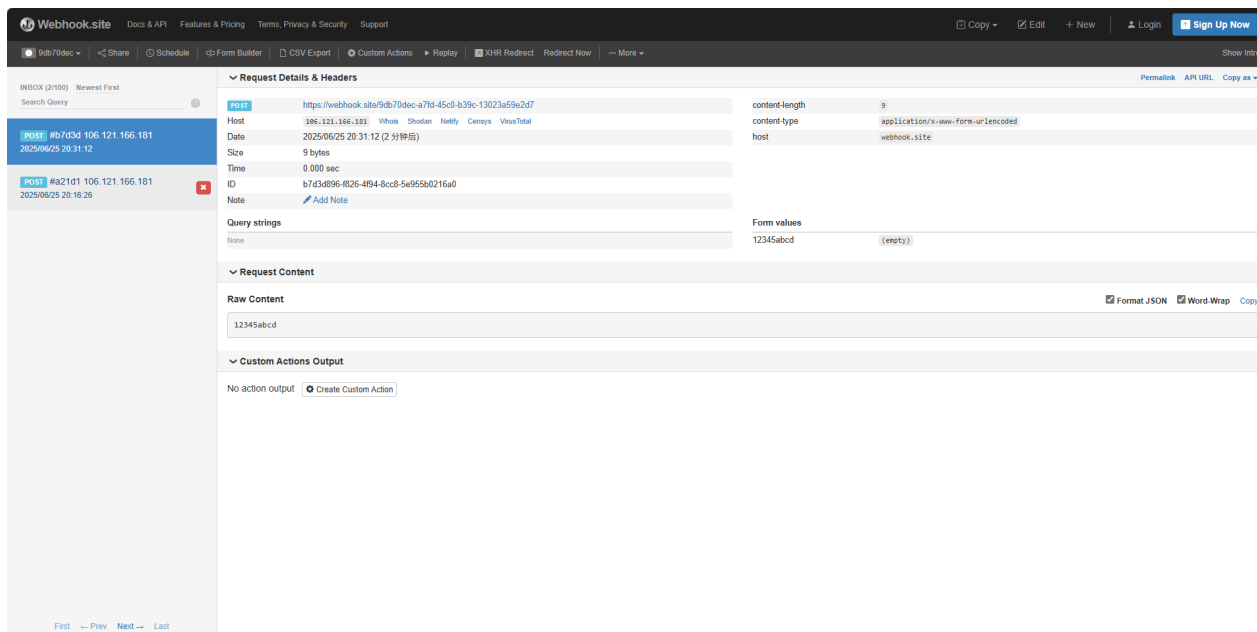
1. 局域网客户端输入字符串并发送：



2. 客户端显示发送成功：



3. Webhook服务器接收到发送的数据：



实现的效果还是不错的，主要是很有趣。

3.2 8266连接WiFi获取信息

8266连接到WiFi网络之后，可以通过其路由器进行互联网访问，之前部分同学可能会使用爬虫进行数据获取，但是爬虫一般习惯使用Python编程。当然8266（至少ESP32可以）其实也可以使用Python编程，但是这就是another story了（有兴趣的同学可以尝试使用[MicroPython编程](#)）。

因此这边我们不会介绍爬虫技术，而是讲解更加简单的调用API+JSON文件处理的方式进行一些特定网站（比如[心知天气](#) or something else）数据的获取和解析。

3.2.1 JSON基础

JSON（JavaScript Object Notation）是一种轻量级的数据交换格式，易于人阅读和编写，同时也易于机器解析和生成。它常用于客户端和服务端之间的数据传输。

JSON的基本语法规则如下：

数据表示：JSON使用键值对的形式来表示数据，类似于Python中的字典或JavaScript中的对象。

- **键 (Key)：**必须是字符串类型，且必须用双引号 (") 括起来。例如： "name" 。
- **值 (Value)：**可以是以下几种类型：
 - **字符串：**用双引号括起来的文本。例如： "John Doe" 。
 - **数字：**整数或浮点数。例如： 42 或 3.14 。
 - **布尔值：** true 或 false （注意大小写）。
 - **数组：**用方括号 ([]) 括起来的有序值集合。例如： [1, 2, 3] 或 ["apple", "banana"] 。
 - **对象：**用花括号 ({ }) 括起来的键值对集合。例如： {"name": "John", "age": 30} 。
 - **null：**表示空值。

举一个JSON片段的例子：

```
{
  "name": "John Doe",
  "age": 30,
  "is_student": false,
  "courses": ["Math", "Science", "History"],
  "address": {
    "street": "123 Main St",
    "city": "Anytown",
    "state": "CA",
    "zip": "12345"
  }
}
```

可以使用 <ArduinoJson.h> 这个库来解析JSON数据。具体内容留待同学们自行探索。

3.2.2 使用8266获取天气信息

这里涉及到API的调用和JSON数据的解析。

因为**涉及民生赛道的部分内容**，此处内容留待同学们**自行探索**，等到硬设结束之后我可能会把这部分内容补全并发布完整版讲义（画个饼x）

3.3 AP模式挂载网页

有了前面的基础知识铺垫，在AP模式下挂载网页也相当的简单了。

我们需要做的步骤如下：

1. 创建WiFi（局域网）
2. 创建Web服务器
3. 设计网页内容
4. 定义响应函数
5. 设置根路径处理函数
6. 启动服务器，处理客户端请求

下面给一个例子来说明：

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

const char *ssid = "ESP8266_AP";      // AP 名称
const char *password = "12345678";    // AP 密码

ESP8266WebServer server(80); // 启动服务器，监听 80 端口

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>ESP8266 AP 模式网页</title>
  <meta charset="utf-8" />
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    .slider {
      width: 100%;
      margin: 20px 0;
    }
    .button {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
    }
  </style>
</head>
```

```

<body>
  <h1>你好！ 欢迎访问ESP8266网页</h1>
  <div>
    <label for="slider">滑动条: </label>
    <input type="range" id="slider" class="slider" min="0" max="100"
value="50" oninput="updateSliderValue()">
    <span id="slider-value">50</span>
  </div>
  <button id="color-button" class="button" onclick="toggleColor()">切换颜色
</button>

  <script>
    function updateSliderValue() {
      var slider = document.getElementById("slider");
      var sliderValue = document.getElementById("slider-value");
      sliderValue.textContent = slider.value;
      // 发送滑动条值到服务器
      fetch('/update_slider', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ value: slider.value })
      });
    }

    function toggleColor() {
      var button = document.getElementById("color-button");
      if (button.style.backgroundColor === "red") {
        button.style.backgroundColor = "blue";
      } else {
        button.style.backgroundColor = "red";
      }
      // 发送按钮颜色到服务器
      fetch('/update_button', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ color: button.style.backgroundColor })
      });
    }
  </script>
</body>
</html>
)rawliteral";

```

```

void handleRoot() {
  server.send(200, "text/html", index_html);
}

```

```

void handleSliderUpdate() {
    if (server.hasArg("plain")) {
        String value = server.arg("plain");
        Serial.print("滑动条值: ");
        Serial.println(value);
    }
    server.send(200, "text/plain", "");
}

void handleButtonUpdate() {
    if (server.hasArg("plain")) {
        String color = server.arg("plain");
        Serial.print("按钮颜色: ");
        Serial.println(color);
    }
    server.send(200, "text/plain", "");
}

void setup() {
    Serial.begin(115200);

    // 启动AP
    WiFi.softAP(ssid, password);
    Serial.println("AP已启动, IP地址为:");
    Serial.println(WiFi.softAPIP());

    // 设置网页路由
    server.on("/", handleRoot);
    server.on("/update_slider", HTTP_POST, handleSliderUpdate);
    server.on("/update_button", HTTP_POST, handleButtonUpdate);

    // 启动Web服务器
    server.begin();
    Serial.println("HTTP服务器已启动");
}

void loop() {
    server.handleClient();
}

```

关键点解析：

结合JS，每次滑动滑动条都会**触发一个 oninput 事件**，这个事件会调用 `updateSliderValue()` 函数，该函数会获取滑动条的值，并更新显示的数值，同时**使用 fetch 方法向服务器发送一个POST请求**，携带滑动条的值。

同样的，在点击按钮时，会触发 `toggleColor()` 函数，该函数会切换按钮的颜色，并**使用 fetch 方法向服务器发送一个POST请求**，携带按钮的颜色。

这里客户端的HTML中内嵌的JS定义的Fetch方法分别是发送给 `/update_slider` 和 `/update_button` 两个路径上的POST请求，而这服务器上配置了对应的处理函数 `handleSliderUpdate()` 和 `handleButtonUpdate()`（配置对应的路由），这两个函数会接收请求并打印出滑动条的值和按钮的颜色。这里的 `hasArg("plain")` 意味着服务器收到的请求中有客户端中的原始数据。

效果是这样的：在客户端滑动滑动条和点击按钮都可以**实时**在服务器串口上看到对应的数值变化



你好！欢迎访问ESP8266网页

滑动条:



51

切换颜色



你好！欢迎访问ESP8266网页

滑动条:



21

切换颜色

```
18:53:02.500 -> 滑动条值: {"value": "43"}
18:53:02.500 -> 滑动条值: {"value": "42"}
18:53:02.531 -> 滑动条值: {"value": "40"}
18:53:02.531 -> 滑动条值: {"value": "38"}
18:53:02.531 -> 滑动条值: {"value": "39"}
18:53:02.566 -> 滑动条值: {"value": "37"}
18:53:02.566 -> 滑动条值: {"value": "36"}
18:53:02.566 -> 滑动条值: {"value": "35"}
18:53:02.599 -> 滑动条值: {"value": "34"}
18:53:02.599 -> 滑动条值: {"value": "33"}
18:53:02.599 -> 滑动条值: {"value": "32"}
18:53:02.634 -> 滑动条值: {"value": "31"}
18:53:02.634 -> 滑动条值: {"value": "30"}
18:53:02.634 -> 滑动条值: {"value": "29"}
18:53:02.634 -> 滑动条值: {"value": "28"}
18:53:09.044 -> 按钮颜色: {"color": "blue"}
18:53:09.259 -> 按钮颜色: {"color": "red"}
18:53:09.536 -> 按钮颜色: {"color": "blue"}
18:53:09.724 -> 按钮颜色: {"color": "red"}
18:53:10.003 -> 按钮颜色: {"color": "blue"}
```

3.4 STA模式挂载网页

STA模式挂载网页过程很像，只是这里我们8266的IP地址不再是自己决定的默认192.168.4.1，而是由连接到的路由器分配的IP地址，这需要我们在8266连接上WiFi之后获取自己的IP地址，接下来的步骤和AP模式是相同的：
这里我们只需要稍微修改一部分代码即可，下面直接给出最需要注意的部分：

```
// AP模式下，下面这段用于查看自己开启的局域网中自己的IP地址
Serial.println(WiFi.softAPIP());

// STA模式下，下面这段用于查看自己连接WiFi网络之后的IP地址
Serial.println(WiFi.localIP());
```

修改之后，我们也需要对应在浏览器中输入STA模式分配的IP地址进行访问。

当然，这样一说我们使用STA只是给我们的IP地址获取带来了麻烦（不是默认，而是需要通过比如串口来读取）吗？当然不是，STA模式的最大好处就是我们可以访问互联网。但是这里很多数据需要API去调用，这块找一个连接稳定+免费的API往往是整个工作中最麻烦的一件事，对不同的要求API也完全不同，这里就留待各位同学自行探索了。

3.5 more dynamics——WebSocket的引入

3.5.1 HTTP的不足——单向性

经典的HTTP是一次性的、是客户端到服务器的单向的，因此如果我们想要实现服务器上更新了的数据也更新到客户端网页上，我们能想到的最经典的做法就是轮询（Polling），即客户端定时向服务器发送请求，询问是否有新的数据。这种方式虽然简单，但会带来以下问题：

1. **延迟**：客户端需要等待下一个轮询周期才能获取到最新数据，这会导致数据更新的延迟。
2. **资源浪费**：即使服务器没有新数据，客户端仍然会发送请求，这会浪费带宽和服务器资源。

而这一切的缘由主要就是HTTP的单向性，服务器更新数据的消息不能由服务器传输到客户端。

3.5.2 全双工协议——WebSocket

这里我们就不再详细介绍WebSocket的协议内容，主要是以一个简单的例子介绍WebSocket的工作逻辑：

下面这个例子中，我们会使用8266的STA模式连接到手机的热点，然后将一个写好的网页挂载到其根路径

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <WebSocketsServer.h>

const char* ssid = "your_SSID";
const char* password = "your_PASSWORD";
```



```

ESP8266WebServer server(80);
WebSocketsServer websocket(81);

const char* sentences[] = {
    "Hello, world!",
    "This is ESP8266.",
    "Welcome to the IoT world.",
    "Have a nice day!",
    "Keep learning and experimenting.",
    "ESP8266 is awesome.",
    "Let's build something cool.",
    "Stay curious.",
    "Happy coding!"
};

const char* htmlContent = R"rawliteral(
<!DOCTYPE html>
<html>
  <body>
    <h1>ESP8266 WebSocket Example</h1>
    <input type="text" id="messageBox" style="width: 300px; height: 50px;
font-size: 16px;" readonly>
    <button onclick="sendMessage()" style="font-size: 16px; padding:
10px;">Get Random Sentence</button>
    <script>
      var socket;
      function initSocket() {
        socket = new WebSocket('ws://%s:81');
        socket.onmessage = function(event) {
          document.getElementById('messageBox').value = event.data;
        };
      }
      function sendMessage() {
        if (!socket) initSocket();
        socket.send('request');
      }
      window.onload = initSocket;
    </script>
  </body>
</html>
)rawliteral";

void handleRoot() {
  String html = htmlContent;
  html.replace("%s", WiFi.localIP().toString());
  server.send(200, "text/html", html);
}

void websocketEvent(uint8_t num, WStype_t type, uint8_t * payload, size_t
length) {

```

```

if (type == WStype_TEXT) {
    if (strcmp((char*)payload, "request") == 0) {
        int randomIndex = random(0, 9);
        String randomSentence = sentences[randomIndex];
        websocket.sendTXT(num, randomSentence.c_str());
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConnected to WiFi");

    server.on("/", handleRoot);
    server.begin();

    websocket.begin();
    websocket.onEvent(webSocketEvent);
}

void loop() {
    server.handleClient();
    websocket.loop();
}

```

`new WebSocket('ws://%s:81')` 是客户端的JS代码，用于初始化建立和8266服务器WebSocket的连接，其中 %s 会被8266的IP地址替换（大家可以发现上面的相关代码）每次点击按钮，会通过socket连接向服务器发送一个文本消息 'request'，然后服务器端根据设置的 `onEvent(xxx Function)` 中的回调函数 `xxx Function` 来处理这个信息，这个回调函数就是代码中的 `webSocketEvent`，注意这里的回调函数参数要求是固定的，可以不用 但是必须有这些，应该也是不能增加的（你可以理解为这是一个函数 `onEvent` 的参数，这个参数作为一个函数，其对应的参数类型是库预先定义好的）处理结果就是随机挑选一个句子作为文本信息发送给客户端。

注意 ⚠： 这里 `num` 可能会让大家感觉很奇怪，这个是WebSocket服务器给每个连接的客户端分配的一个唯一标识，用于回复信息使用。

附录D部分我会附上在写这个讲义早期（大概前天）使用的一个WebSocket Demo，我觉得也相当的有趣，实现了多机间的通信，是可以类比实现一个局域网论坛网站形式的。大家有兴趣可以尝试一下。

附录

A.一次不怎么成功的实验结果——尝试通过软件让ESP8266实现“内网穿透-like”效果

ESP8266本身并不支持直接的网络共享功能，因此我们无法直接让8266实现类似于内网穿透的效果。但是我当时根据整个联网流程设想了一个软件方案：我每次向 192.168.4.1 请求一个url的访问，比如 `www.baidu.com`，8266作为局域网服务器收到这个请求会通过连接的路由器访问 `www.baidu.com`，如果按照我们之前介绍的，这本质上就是从服务器获取一些文件

(html、css、js e.t.c)，因此我们可以利用8266进行一次转发，将这些文件转发到局域网的客户端上，从而实现类似内网穿透的效果。

在实践之前，我们可以推测一下最终的效果，首先如果有其他机制的话，HTML文件的转发以及渲染是正常的，因此我们可以期待看到经典的Baidu-like的搜索框。但是显然很多的链接是需要实时更新的，这些链接或者一些LOGO图片显然不会是缓存在HTML文件中，而是通过网络链接来获取的，但是因为我们手机客户端毕竟还是一个局域网，是无法访问这些链接的，所以我们大概率是无法看到LOGO图片的。

最终效果确实如我们所推测的那样：



对应的测试代码如下：

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <WiFiClient.h>

const char* staSSID = "ESP8266_TEST_WIFI"; // 替换为你的路由器WiFi名称
const char* staPassword = "qasd11112222"; // 替换为你的路由器WiFi密码
const char* apSSID = "ESP8266_AP"; // AP模式下的WiFi名称
const char* apPassword = "12345678"; // AP模式下的WiFi密码

ESP8266WebServer server(80); // 创建一个Web服务器对象，端口为80
```

```

void handleRoot() {
    String message = "Hello, this is ESP8266 AP. Your IP address is: ";
    message += server.client().remoteIP().toString();
    server.send(200, "text/plain", message);
}

void handleRequest() {
    // 获取请求的URL
    String url = server.arg("url");
    if (url.length() == 0) {
        server.send(400, "text/plain", "Bad Request: URL parameter missing");
        return;
    }
    // 创建一个客户端连接到请求的URL
    WiFiClient client;
    if (!client.connect(url.c_str(), 80)) {
        server.send(500, "text/plain", "Internal Server Error: Unable to connect to URL");
        return;
    }
    // 发送HTTP GET请求
    client.println("GET / HTTP/1.1");
    client.println("Host: " + url);
    client.println("Connection: close");
    client.println();

    // 读取响应并发送回客户端
    String response = "";
    bool headersDone = false;
    while (client.connected()) {
        String line = client.readStringUntil('\n');
        if (line == "\r") {
            headersDone = true; // 结束头部信息
            continue;
        }
        if (!headersDone) {
            response += line + "\n";
        } else {
            response += line;
        }
    }
    server.send(200, "text/html", response);
}

void setup() {
    Serial.begin(115200);
    delay(10);
    // 设置为AP+STA模式
    WiFi.mode(WIFI_AP_STA);
}

```

```
// 连接到STA模式的WiFi网络
WiFi.begin(staSSID, staPassword);
Serial.print("Connecting to STA WiFi...");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nConnected to STA WiFi!");
Serial.print("STA IP address: ");
Serial.println(WiFi.localIP());
// 创建AP模式的WiFi网络
WiFi.softAP(apSSID, apPassword);
Serial.print("AP mode started. AP IP address: ");
Serial.println(WiFi.softAPIP());

// 设置Web服务器的根路径处理函数
server.on("/", handleRoot);
server.on("/request", handleRequest);
server.begin(); // 启动Web服务器
}

void loop() {
    server.handleClient(); // 处理Web服务器的客户端请求
}
```

但是因为算法设计的不妥之处，我们会遇到一些问题：第一，在访问 `.../request?www.baidu.com` 的时候，刷新网页，渲染出的结果都会很不一样，第二，除了 `www.baidu.com`，我还真没找到另外允许实现这种转发的网站。（会报各种错误）

#Notes 其实这里我们就歪打误撞half-create了正向Proxy的逻辑：

正向代理（Forward Proxy） 是指客户端通过代理服务器访问目标网站。代理服务器充当客户端与目标服务器之间的中介。

但是 ⚠️ 和专业 Proxy 的差异

- **简化的功能：**
 - 只处理了 GET 请求
 - 不支持 HTTPS（无法代理百度、知乎等现代网站）
 - 不支持 chunked 编码或 gzip 压缩
 - 不支持缓存、认证、日志记录等特性
 - **不具备“透明代理”功能：**
 - 必须显式通过 `/request?url=` 提交目标网址
 - 并非标准代理协议（如 SOCKS、HTTP CONNECT）
- 一方面是目前的算法设计不足，另一方面是**8266自身的限制**：

1. 不支持 HTTPS（SSL/TLS）访问：

- ESP8266 默认的 `WiFiClient` 只能处理 **HTTP 明文协议**。

- 要访问 `https://www.zhihu.com` 这样的站点，需要 `WiFiClientSecure`。但即使使用 `WiFiClientSecure`，也会遇到这些问题：
- 内存不足导致 TLS 握手失败；
- 要禁用证书验证（不安全）；
- 很多网站使用 **SNI (Server Name Indication)**，ESP8266 支持有限。

2. 大部分网站返回的内容都有 `Content-Encoding: gzip` 或 `Transfer-Encoding: chunked`，如果直接转发，浏览器将：

- 无法解码；
- 渲染失败；
- 显示乱码或白屏。

同学们对Proxy感兴趣，可以尝试使用**ESP32**进行更加稳定完善的功能实现，这里我就不继续探索了。

B. AP+STA模式代码

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <WiFiClientSecure.h> // 包含SSL支持

const char* staSSID = "ESP8266_TEST_WIFI"; // 替换为你的路由器WiFi名称
const char* staPassword = "qasd11112222"; // 替换为你的路由器WiFi密码
const char* apSSID = "ESP8266_AP"; // AP模式下的WiFi名称
const char* apPassword = "12345678"; // AP模式下的WiFi密码

const char* webhookURL = "webhook.site"; // Webhook域名
const char* webhookPath = "your_Webhook_path"; // Webhook路径（替换为自己的Webhook path）

ESP8266WebServer server(80); // 创建一个Web服务器对象，端口为80
void handleRoot() {
    String html = R"rawliteral(
        <!DOCTYPE html>
        <html>
        <head>
            <title>Send Data to Webhook</title>
        </head>
        <body>
            <h1>Send Data to Webhook</h1>
            <form action="/send" method="post">
                <label for="data">Data:</label><br>
                <input type="text" id="data" name="data"><br><br>
                <input type="submit" value="Submit">
            </form>
        </body>
    )rawliteral";
    server.send(200, "text/html", html);
}
```

```

    </html>
)rawliteral";
server.send(200, "text/html", html);
}

void handleSend() {
    if (server.method() == HTTP_POST) {
        String data = server.arg("data");
        if (data.length() > 0) {
            // 发送数据到Webhook
            WiFiClientSecure client;
            client.setInsecure(); // 跳过证书验证
            Serial.println("Connecting to Webhook server...");
            if (client.connect(webhookURL, 443)) {
                Serial.println("Connected to Webhook server");
                client.println("POST " + String(webhookPath) + " HTTP/1.1");
                client.println("Host: " + String(webhookURL));
                client.println("Content-Type: application/x-www-form-urlencoded");
                client.print("Content-Length: ");
                client.println(data.length());
                client.println();
                client.println(data);

                // 读取响应
                String response = "";
                while (client.available()) {
                    response += (char)client.read();
                }
                server.send(200, "text/html", "Data sent to Webhook. Server
response: " + response);
            } else {
                Serial.println("Failed to connect to Webhook server");
                server.send(500, "text/html", "Failed to connect to Webhook
server");
            }
        } else {
            server.send(400, "text/html", "No data received");
        }
    } else {
        server.send(405, "text/html", "Method Not Allowed");
    }
}

void setup() {
    Serial.begin(115200);
    delay(10);

    // 设置为AP+STA模式
    WiFi.mode(WIFI_AP_STA);
    // 连接到STA模式的WiFi网络

```



```
WiFi.begin(staSSID, staPassword);
Serial.print("Connecting to STA WiFi...");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nConnected to STA WiFi!");
Serial.print("STA IP address: ");
Serial.println(WiFi.localIP());
// 创建AP模式的WiFi网络
WiFi.softAP(apSSID, apPassword);
Serial.print("AP mode started. AP IP address: ");
Serial.println(WiFi.softAPIP());

// 设置Web服务器的根路径处理函数
server.on("/", HTTP_GET, handleRoot);
server.on("/send", HTTP_POST, handleSend);
server.begin(); // 启动Web服务器
Serial.println("Web server started");
}

void loop() {
    server.handleClient(); // 处理Web服务器的客户端请求
}
```

C. 状态码详细介绍

状态码类别	状态码	状态描述	详细说明
1xx 信息性状态码	100	Continue	请求已接收，客户端应继续发送请求剩余部分（如POST数据）。很少用。
	101	Switching Protocols	服务器正在切换协议，响应客户端请求切换（如HTTP升级为WebSocket）。
2xx 成功状态码	200	OK	请求成功，响应包含请求的资源。
	201	Created	请求成功，服务器已创建新的资源（常用于POST请求）。
	202	Accepted	请求已接受，但尚未处理完成。
	204	No Content	请求成功，但无响应体内容（常用于DELETE请求响应）。
3xx 重定向状态码	300	Multiple Choices	请求有多个可能响应，客户端可选择。
	301	Moved Permanently	资源已永久移动，客户端以后应使用新地址访问。

状态码类别	状态码	状态描述	详细说明
	302	Found	资源临时移动，客户端仍使用原请求地址。
	303	See Other	请求应使用GET方法访问另一个URI。
	304	Not Modified	资源未修改，客户端可使用缓存内容，无需重新下载。
	307	Temporary Redirect	临时重定向，客户端应保持原方法重新请求新地址。
4xx 客户端错误	400	Bad Request	请求格式错误或语法错误，服务器无法理解。
	401	Unauthorized	请求未授权，需进行身份验证（登录）。
	403	Forbidden	服务器拒绝请求，客户端无权访问资源。
	404	Not Found	请求的资源不存在。
	405	Method Not Allowed	请求方法不被允许（如GET请求被禁止，服务器只允许POST）。
	408	Request Timeout	请求超时，服务器关闭连接。
	429	Too Many Requests	客户端发送请求过多，超过服务器限制（限流）。
5xx 服务器错误	500	Internal Server Error	服务器内部错误，无法完成请求。
	501	Not Implemented	服务器不支持请求的方法。
	502	Bad Gateway	服务器作为网关或代理时收到无效响应。
	503	Service Unavailable	服务器当前不可用，可能是维护或过载。
	504	Gateway Timeout	服务器作为网关或代理时未及时收到上游服务器响应。

D. WebSocket Demo

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <WebSocketsServer.h>

const char* ssid = "ESP8266_AccessPoint";
const char* password = "12345678";

ESP8266WebServer server(80);
WebSocketsServer websocket = WebSocketsServer(81);

int counter = 0;
```

```

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE HTML>
<html>
  <head>
    <title>ESP8266 Web Server</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        margin: 0;
        padding: 0;
        background-color: #f4f4f4;
      }
      .container {
        width: 80%;
        margin: 50px auto;
        padding: 20px;
        background: #fff;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      }
      h1 {
        color: #333;
      }
      p {
        font-size: 18px;
        color: #666;
      }
      button {
        background-color: #4CAF50;
        color: white;
        padding: 10px 20px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        font-size: 16px;
      }
      button:hover {
        background-color: #45a049;
      }
    </style>
  </head>
  <body>
    <div class="container">
      <h1>Hello from ESP8266!</h1>
      <p id="message">Counter: <span id="counter">0</span></p>
      <button onclick="incrementCounter()">Click Me</button>
    </div>
    <script>
      var ws = new WebSocket('ws://' + window.location.hostname + ':81');
      ws.onmessage = function(event) {
        document.getElementById('counter').innerText = event.data;

```

```

    };
    function incrementCounter() {
        fetch('/increment')
            .then(response => response.text())
            .then(data => {
                document.getElementById('counter').innerText = data;
            });
    }
</script>
</body>
</html>
)rawliteral";

void handleRoot() {
    server.send(200, "text/html", index_html);
}

void handleIncrement() {
    counter++;
    String counterStr = String(counter); // 创建一个String对象
    websocket.broadcastTXT(counterStr); // 将String对象传递给broadcastTXT
    server.send(200, "text/plain", counterStr);
}

void websocketEvent(uint8_t num, WStype_t type, uint8_t *payload, size_t
length) {
    switch (type) {
        case WStype_DISCONNECTED:
            Serial.printf("[%u] Disconnected!\n", num);
            break;
        case WStype_CONNECTED:
            Serial.printf("[%u] Connected!\n", num);
            break;
        case WStype_TEXT:
            Serial.printf("[%u] get Text: %s\n", num, payload);
            break;
    }
}

void setup() {
    Serial.begin(115200);
    delay(10);
    WiFi.softAP(ssid, password);
    Serial.println("SoftAP started");
    Serial.print("SoftAP IP address: ");
    Serial.println(WiFi.softAPIP());
    server.on("/", handleRoot);
    server.on("/increment", handleIncrement);
    server.begin();
    Serial.println("HTTP server started");
}

```

```
websocket.begin();  
websocket.onEvent(webSocketEvent);  
}  
  
void loop() {  
  server.handleClient();  
  websocket.loop();  
}
```